
ProofCouncil: An LLM Agent for Solving Open Mathematical Problems

Johannes Schmitt¹, Tim Gehringer¹, Jasper Dekoninck¹,
Gergely Berczi², Uri Kreitner¹, Liam Price³
¹ETH Zurich, ²Aarhus University, ³Independent Researcher
Correspondence: johannes.schmitt@math.ethz.ch

 <https://github.com/eth-sri/proof-council>

Abstract

Large language models (LLMs) have shown increasing promise in solving open problems in mathematics. However, their performance can be further improved through agentic workflows tailored to real-world mathematical practice. To this end, we introduce ProofCouncil, a mathematical agent that is designed to tackle open problems using an author-critic architecture. ProofCouncil served as a submission to the second batch of FirstProof, a challenge consisting of 10 real-world mathematical problems that agents must solve autonomously. Its submissions for 6 of the 10 problems were judged by the referees to be correct up to at most minor revisions, showing the best performance among participating teams. We also evaluate ProofCouncil on 30 open problems collected from mathematical researchers. Among the 17 solutions that received human feedback, 4 were judged completely correct, and a further 9 contained useful partial progress. In this short paper, we describe the development of ProofCouncil and the agent-building library used to create it, which we release as open source to the community.

1 Introduction

Large language models (LLMs) have recently demonstrated remarkable capabilities in mathematics by solving open problems of varying interest (Alon et al., 2026; Schmitt, 2025; Dixit et al., 2026). This progress has been accompanied by the development of advanced agents designed specifically for mathematical problem-solving (Dixit et al., 2026; Zhao et al., 2026; Zheng et al., 2026; Ju et al., 2026), further highlighting the potential of LLMs in this domain. In this context, the second iteration of the FirstProof challenge (Abouzaid et al., 2026) was launched to evaluate AI systems on a fixed set of 10 problems with no publicly available solution. Participants were required to submit an agent that autonomously attempted all 10 problems within a 24-hour window, using only publicly available models and tools.

In this work, we present ProofCouncil, a mathematical agent submitted to the FirstProof challenge. As shown in Fig. 1, ProofCouncil follows an author-critic architecture: an author agent iteratively writes and revises a proof, while a critic agent evaluates each version and provides feedback. At every round, the author may also request targeted assistance from two auxiliary sources: a council of additional LLMs outside the main author-critic loop, and a compute agent for computer algebra system (CAS) computations. Their responses are provided to the author in the next iteration. To maintain consistency and allow the critic to track progress, the critic generally retains its conversation history across rounds, but this history is reset every k rounds to provide a more independent review. If a freshly initialized critic accepts the proof, the proof is returned to the user.

We open-source both ProofCouncil and the underlying agent-building library. The library represents agentic systems as conditional directed acyclic graphs (DAGs), enabling flexible structures of calls between different agents and models. It also provides a simple interface for

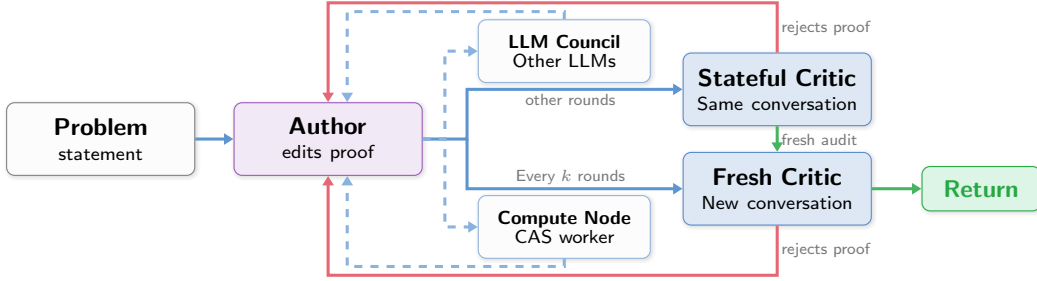


Figure 1: Overview of ProofCouncil. The author agent iteratively edits the proof in response to feedback from a critic. Every k rounds, the stateful critic is reset. The author may optionally request help from other LLMs or a compute node.

implementing new agents and workflows, enabling future researchers to build on our work and create their own agentic systems.

In the FirstProof challenge, ProofCouncil solved 6 of the 10 problems up to at most minor revisions and showed the best performance among participating teams. As a secondary evaluation, we also reached out to mathematical researchers at ETH Zurich and collected 30 open problems across various fields of mathematics, with submissions coming both from ETH and, via private communication, from researchers at a few other institutions. Solution attempts were sent to the question authors for feedback. Among 17 received responses, 4 were judged to be correct solutions, with 9 more containing useful partial progress towards a solution. Further, among the received reviews, no expert reported that an output claiming to solve the stated problem was mathematically false; however, two outputs were judged to have solved easier interpretations of the submitted problem, highlighting problem interpretation as a notable failure mode. We do not release the solutions to these researcher-provided problems, as they are the intellectual property of the researchers who shared them with us and may be used by them at their discretion.

Our key contributions are as follows:

- We introduce ProofCouncil, a mathematical agent designed to solve complex problems through an author-critic architecture (Sec. 2.1).
- We develop and release an open-source agent-building library that enables flexible and efficient construction of agentic systems as conditional DAGs (Sec. 2.2).
- We demonstrate the effectiveness of ProofCouncil by solving 6 out of 10 problems (up to at most minor revisions) in the FirstProof challenge and producing correct solutions to 4 open problems collected from mathematical researchers (Sec. 3).

2 ProofCouncil

In this section, we first provide an overview of the system architecture of ProofCouncil, then discuss the library we developed to build ProofCouncil as conditional DAGs of agent calls. The prompts are provided in App. C.

2.1 ProofCouncil Workflow

The overall workflow of ProofCouncil is shown in Fig. 1. The system proceeds iteratively until one of four stopping conditions is met: a solution is accepted, the maximum number of rounds is reached, the total cost budget is exhausted, or the timeout is reached. We describe each component in turn.

Author The author is responsible for producing and revising the proof. It uses GPT-5.5-Pro (xhigh) with access to built-in code execution and web search tools. Through code execution, the author maintains and edits three files:

- `answer.tex`: the main $\text{\LaTeX}$ file containing the proof.
- `research_notes.tex`: a scratchpad for notes, ideas, failed attempts, and partial results that are not part of the main proof but may help the author or critic track progress.
- `references.bib`: a BibTeX file containing the references cited in the proof.

At each iteration, the author receives the latest critic feedback, any auxiliary information produced by the council or compute node, and the current state of all files. It then updates the proof and, when useful, its research notes and bibliography.

The author can also request auxiliary help by emitting structured XML tags in its output. A `<council>` tag contains a specific question to the LLM council, while a `<compute_agent>` tag contains a question for the compute node. The resulting responses are passed back to the author in the next iteration. Finally, the author must emit a `<ready>` tag indicating whether it believes it has found a complete and correct solution.

Critic The critic is responsible for assessing the correctness of the current proof. It receives the files written by the author and provides feedback on gaps, errors, or missing justifications. Like the author, it uses GPT-5.5-Pro (xhigh) with access to the same tools.

By default, the critic is stateful. Each new version of the proof is appended to the critic’s conversation history, allowing the critic to provide consistent feedback. To reduce path dependence and obtain more independent feedback, we reset the critic every k rounds. In our experiments, we use $k = 3$.

The critic also emits an `<answer_ready>` tag indicating whether it considers the proof complete and correct. If the stateful critic accepts the proof, a fresh critic is called for an additional audit. The proof is returned as a solution only if the fresh critic also accepts it, and the author has marked the answer as ready.

LLM council The LLM council is an optional auxiliary component that provides additional feedback outside the main author-critic loop. When called, each council member receives all files written by the author, together with the specific question posed by the author. The council currently consists of Gemini-3.1-Pro (high), Claude-4.7-Opus (Max), and GPT-5.5-Pro (xhigh). The models answer independently, without seeing one another’s responses. Their answers are then given to the author as additional context in the next iteration.

Compute node The compute node is an optional auxiliary component for code execution and computer algebra system (CAS) computations. It is implemented as a Codex agent using GPT-5.5 (xhigh), with access to specialized mathematical software including SageMath (The SageMath Developers, 2026), GAP (GAP, 2026), Singular (Decker et al., 2024), and Pari (The, 2023). The compute node receives all files written by the author, together with the author’s specific question and returns a response that may include code, CAS output, or explanatory analysis. This response is provided to the author in the next iteration. In addition, the edited code repository is added to the file system available to the author, allowing it to inspect or reuse the compute node’s outputs in subsequent reasoning.

2.2 Agents as DAGs

To create ProofCouncil, we developed an open-source library for constructing agentic systems as conditional directed acyclic graphs (DAGs). We briefly describe its main features here.

Agents as DAGs In our library, an agent is represented as a DAG whose nodes can be LLM calls, custom Python blocks, CLI-based compute agents, deterministic gates (such as compilation checks), or control-flow blocks, with edges encoding the dependencies between them. Nodes may themselves be agents, enabling hierarchical compositions of models, tools, and workflows. An LLM node, for instance, is defined by parameters such as a prompt template, a model name, and a set of outputs. The prompt is rendered at runtime using the outputs of upstream nodes, and the outputs produced by a node can in turn serve as inputs to downstream nodes. This abstraction supports complex information flow while keeping the structure of the system explicit.

Conditional execution Plain DAGs are not expressive enough for many agentic workflows since the set of calls to execute may depend on intermediate outputs. For instance, in ProofCouncil, the LLM council is called only when the author emits a corresponding tag. To support this behavior, edges can be equipped with conditions that determine whether a downstream node should execute based on the outputs of its parents.

Although the workflow in Fig. 1 contains a feedback loop from the critic to the author, it can be represented as a conditional DAG by unrolling the loop for a fixed number of rounds. For readability, the library supports bounded repeat blocks that represent loops, producing a structure that is easier to read and edit while still executing as a finite conditional DAG.

Agent execution To execute an agent, the library computes a topological order of the nodes in the DAG. A node is run as soon as all of its dependencies have completed and its edge conditions are satisfied. This enables independent nodes to execute in parallel, improving efficiency in workflows with multiple auxiliary calls or independent branches.

User interface We also built a custom user interface for editing and running agents, inspired by Blender nodes (Blender Online Community, 2026). The interface allows users to visually construct and modify the DAG structure, edit prompt templates, add or remove outputs, and monitor the execution of each node. Outputs are specified by requiring models to emit structured YAML tags. Custom components can be added as nodes, and control-flow constructs such as conditionals and repeat blocks can be created visually. Screenshots are provided in App. B.

3 Evaluation

We evaluate ProofCouncil on two sets of problems: a set of open problems collected from mathematical researchers, primarily at ETH Zurich, and the 10 problems from FirstProof. The former set was collected to test the system on a broader and more diverse collection of research problems. We also used feedback from these runs to improve ProofCouncil.

3.1 Results on Open Questions

To prepare our submission for the FirstProof challenge, we ran several preliminary evaluations: three single-problem runs and three larger runs of 10 problems each. In this section, we report the results of these runs and summarize the feedback we received from human mathematicians on the solutions produced by ProofCouncil.

Problem collection To collect problems, we contacted all members of the mathematics department at ETH Zurich and asked them to share open problems they were interested in seeing attempted by an AI system; further problems reached us from researchers at a few other institutions via private communication. Each problem was screened using GPT-5.5-Pro to check whether it was self-contained and accurately stated. When necessary, edits were made in collaboration with the submitting researcher. The screening prompt is provided in App. C. Across all runs, we evaluated ProofCouncil on 33 distinct problems: 30 problems collected from researchers and 3 public Erdős problems. In App. A, we give a self-contained account of one Erdős-problem result from these runs.

Problem execution Each run used the same codebase, with minor prompt changes and bug fixes introduced between runs based on earlier results. Importantly, three problems in the first 10-problem run encountered execution errors, and we therefore exclude them from the results. In the final 10-problem run, we increased the maximum number of iterations from 10 to 200, effectively allowing the system to run until the 24-hour time limit was reached.

Main results We requested feedback from problem authors on ProofCouncil’s solutions to all non-Erdős problems (shown in Tab. 1). Among the 17 responses received so far, 4

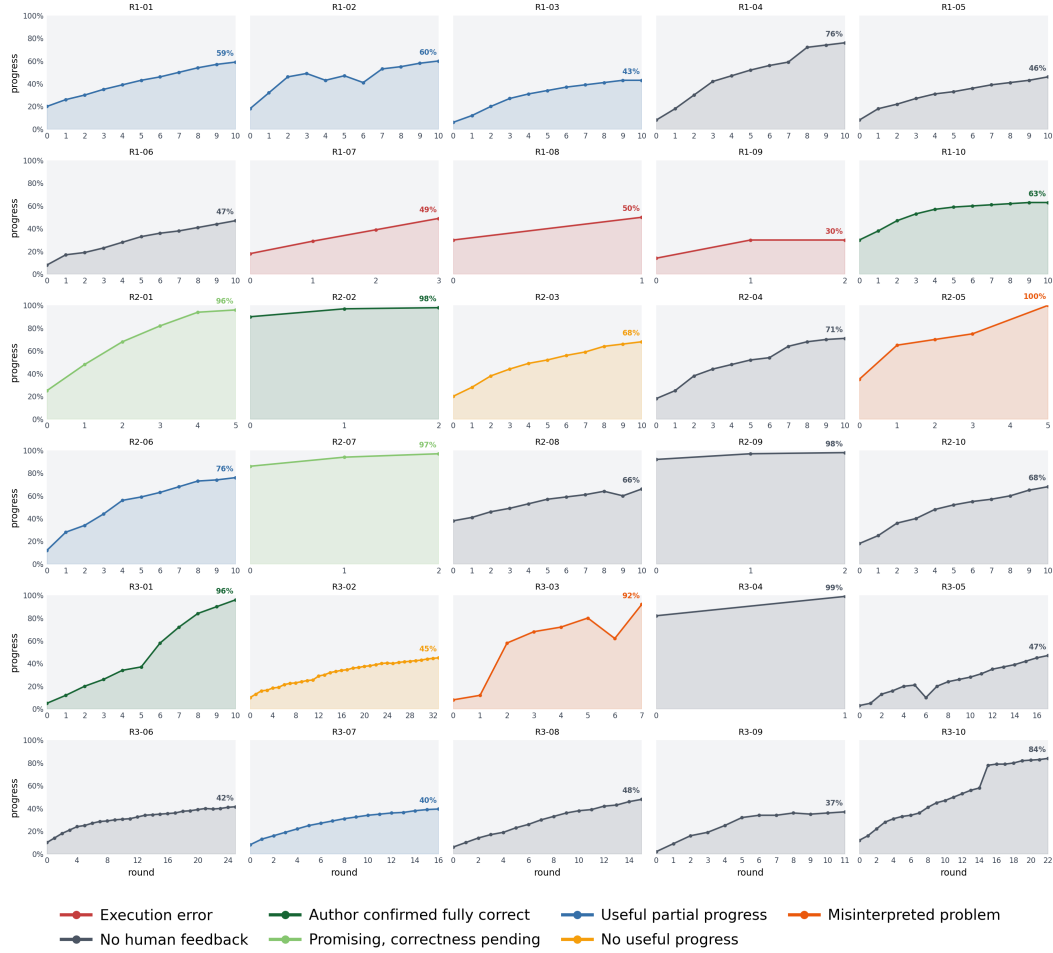


Figure 2: Progress panel for all 30 problems from the three test runs. Curves show GPT-5.5-Pro postscreen progress estimates. Colors indicate final status per human feedback. This analysis only includes the three main 10-problem runs.

Table 1: Summary of human feedback on the open-question evaluation.

Category	Count
Problems evaluated	30
Researcher reviews received	17
Complete solutions	4
Possibly complete solutions	2
Meaningful partial progress	7
No errors, but no progress	2
Misinterpreted problems	2

outputs were judged to be complete solutions,¹ and 2 were considered promising but not yet fully verified by the researchers. Most of the remaining outputs were not complete, but still useful: 7 contained partial progress, while 2 contained no apparent errors but offered little substantive progress. The remaining 2 responses had misinterpreted the problem, producing correct solutions to easier versions of the stated problems. Importantly, among

¹One of these four problems was submitted by an author of this paper. Since the submitter’s own assessment would not be independent, the solution was instead circulated to three external experts in the area, who judged the results new and the argument likely correct; a detailed final verification is still in progress.

the 17 received reviews, no reviewer reported that an output claiming to solve the stated problem was mathematically false; the two misinterpretation cases instead highlight problem interpretation as a significant failure mode. The recorded cost was approximately \$213 per attempted problem. Since responses are still arriving and several researchers have announced more detailed verdicts after a thorough check, we plan to update the results in this section as further feedback comes in.

Qualitative human feedback Several researchers expressed surprise at the quality of the outputs, noting that solutions were useful, constrained the search space for future work, or took the right approach across all subquestions. In one notable case, ProofCouncil found a complete solution using an unexpected approach and went substantially further than the researcher had anticipated for the project. At the same time, researchers also identified important limitations. Some outputs relied on an unorthodox interpretation that made the problem easier or nearly trivial. Others mainly explained why the problem was difficult without making substantial progress. In some cases, the mathematical content was promising, but the write-up did not read like a polished research paper.

Postscreen audit To further analyze the runs, we asked GPT-5.5-Pro to review the intermediate logs and assign a progress score to each problem at each round. The resulting progress panel is shown in Fig. 2. The audit suggests that some problems were only marked as solved after several rounds of feedback, indicating that the iterative author-critic process was important in reaching the final solution. However, this analysis was solely based on the model’s interpretation of the logs, and the results should be interpreted with caution.

The audit also revealed recurring failure modes. In particular, ProofCouncil often became stuck in a local minimum, repeatedly attempting to patch a specific issue in the proof without making meaningful progress toward a complete solution. One possible way to address this would be to run several author-critic threads in parallel, allowing the system to explore more diverse proof strategies, but this would substantially increase cost. Finally, we note that both auxiliary components were used extensively: the compute node and the LLM council each appeared in more than 50% of rounds.

3.2 Results on FirstProof

Main results ProofCouncil attempted all 10 competition problems fully autonomously within the 24-hour window, and the submitted solutions were graded by expert referees appointed by the organizers (in the official FirstProof Second Batch report, our submitted harness appears as System A / IMProofBench ProofStack; this is the submitted version of ProofCouncil). Based on the referee recommendations, the submissions for 6 of the 10 problems (P1, P2, P3, P5, P7, and P9) were judged correct up to at most minor revisions, and one further submission (P10) contained progress towards a solution but requires major revisions. Of the remaining three problems, the submission for P8 was rejected, P4 saw no substantive progress, and for P6 the system encountered a technical failure and submitted no solution. The latter is particularly unfortunate, as P6 can be solved by a single query to GPT-5.5-Pro. Overall, ProofCouncil showed the best performance among the participating teams. We plan to update this paper with a more fine-grained analysis of the logs of this run in the coming days.

4 Limitations and Future Work

We briefly discuss several limitations of our work and directions for future work.

Cost Running ProofCouncil is expensive, with recorded costs exceeding \$200 per attempted problem. This cost is primarily driven by the repeated use of frontier models for both proof generation and verification, as well as by calls to auxiliary agents such as the LLM council and compute node. Reducing this cost is an important direction for future work. Possible approaches include improving stopping criteria or routing simpler subtasks to cheaper models.

Human readability ProofCouncil was optimized primarily for finding correct solutions rather than for producing polished mathematical proofs. Several mathematicians noted that the resulting proofs were dense and would need to be expanded before they could be read by non-experts in the relevant subfield. Improving readability is therefore another important direction for future work. This could involve adding dedicated exposition agents that expand arguments, supply missing background, standardize notation, and rewrite final solutions into a form closer to a research paper.

5 Conclusion

In this work, we introduced ProofCouncil, an agentic system for solving open mathematical problems. ProofCouncil solved 6 of the 10 problems in the second batch of the FirstProof challenge up to at most minor revisions, showing the best performance among participating teams. On a separate set of open problems collected from mathematical researchers, ProofCouncil produced four complete solutions and meaningful partial progress in many other cases. To support further work, we release both ProofCouncil and the underlying open-source library for constructing agentic systems as conditional DAGs.

Use of AI

We describe how AI was used in this project, including the models and tools involved. The code for ProofCouncil was generated with a combination of Claude Code and Codex, guided by human feedback on what should be generated, how ProofCouncil should be structured, and how the interface and generation process should be executed. GPT-5.5-Pro was used to produce the analysis shown in Fig. 2. Its feedback helped us interpret the runs, identify failure modes, and make the necessary changes. In the paper itself, AI was used only to paraphrase paragraphs for clarity and assist with formatting, while all substantive content was written by humans. In addition, Claude Fable 5 (via Claude Code) was used to review the draft of the paper and make some targeted edits following detailed author prompting. Another exception is Sec. A.1, which is a cleaned-up example output from ProofCouncil.

Acknowledgments

We thank the organizers of the FirstProof challenge for hosting the competition and providing a platform for evaluating mathematical problem-solving agents. We also thank the researchers at ETH Zurich and other institutions who shared their open problems with us and provided feedback on our solutions, and Jeremy Feusi for helpful discussions and input in the early phase of the project.

This work was supported by an unrestricted gift from Google Ireland Limited to ETH Zurich, supporting work related to “Research on Evals”, administered through the ETH Zurich Foundation, and by the Swiss National Science Foundation grant 10009122, “Beyond Benchmark Scores: Analyzing AI Reasoning on Research-Level Mathematics”. We also received free API credits for the Gemini API, enabling the evaluation of Gemini models; we thank the team at Google for this contribution.

Contributions

- Johannes Schmitt led the team, designed and implemented the overall system architecture, and served as the primary point of contact for the ETH math researchers and the organizers of the FirstProof challenge.
- Tim Gehringer co-developed the agent-building library, helping Johannes implement the ProofCouncil system and set up the evaluation pipeline.
- Jasper Dekoninck helped design the user interface for the library, and served as the main writer of the paper.

- Gergely Berczi tested the agent, implemented small features in the library, and provided feedback on the paper.
- Uri Kreitner tested the agent and provided feedback on the paper.
- Liam Price tested the agent and provided feedback on the paper.

References

- Mohammed Abouzaid, Andrew J. Blumberg, Martin Hairer, Joe Kileel, Tamara G. Kolda, Paul D. Nelson, Daniel Spielman, Nikhil Srivastava, Rachel A. Ward, Shmuel Weinberger, and Lauren Williams. First proof. *CoRR*, abs/2602.05192, 2026. doi: 10.48550/ARXIV.2602.05192. URL <https://doi.org/10.48550/arXiv.2602.05192>.
- Noga Alon, Thomas F Bloom, WT Gowers, Daniel Litt, Will Sawin, Arul Shankar, Jacob Tsimmerman, Victor Wang, and Melanie Matchett Wood. Remarks on the disproof of the unit distance conjecture. *arXiv preprint arXiv:2605.20695*, 2026.
- Blender Online Community. *Blender: A 3D Modelling and Rendering Package*. Blender Foundation, Amsterdam, The Netherlands, 2026. URL <https://www.blender.org>.
- Thomas F. Bloom. Erdős problem #539. <https://www.erdosproblems.com/539>, 2026. Accessed 9 June 2026.
- Wolfram Decker, Gert-Martin Greuel, Gerhard Pfister, and Hans Schönemann. SINGULAR 4-4-0 — A computer algebra system for polynomial computations. <http://www.singular.uni-kl.de>, 2024.
- Aradhya Dixit, Tianxi Liang, and Jai Telang. Project aletheia: Verifier-guided distillation of backtracking for small language models. *CoRR*, abs/2601.14290, 2026. doi: 10.48550/ARXIV.2601.14290. URL <https://doi.org/10.48550/arXiv.2601.14290>.
- GAP – Groups, Algorithms, and Programming, Version 4.16.0. The GAP Group, 2026. URL <https://www.gap-system.org>.
- Andrew Granville and Friedrich Roesler. The set of differences of a given set. *The American Mathematical Monthly*, 106(4):338–344, 1999. doi: 10.1080/00029890.1999.12005050.
- Ron Holzman, Vsevolod F. Lev, and Rom Pinchasi. Projecting difference sets on the positive orthant. *Combinatorics, Probability and Computing*, 17(5):681–688, 2008. doi: 10.1017/S0963548308009139.
- Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, Zeming Sun, Leheng Chen, Yutong Wang, Yuefeng Wang, Zichen Wang, Wanyi He, et al. Automated conjecture resolution with formal verification. *arXiv preprint arXiv:2604.03789*, 2026.
- Imre Leader. Positive projections. DIMAP Seminar abstract, University of Warwick, 2009. URL https://warwick.ac.uk/fac/cross_fac/dimap/seminars/. 23 June 2009; joint work with Béla Bollobás.
- Imre Leader. Positive projections. Combinatorial Theory Seminar abstract, Mathematical Institute, University of Oxford, 2012. URL <https://www.maths.ox.ac.uk/node/9547>. 7 February 2012; joint work with Béla Bollobás.
- Johannes Schmitt. Extremal descendant integrals on moduli spaces of curves: An inequality discovered and proved in collaboration with ai. *arXiv preprint arXiv:2512.14575*, abs/2512.14575, 2025. doi: 10.48550/ARXIV.2512.14575. URL <https://arxiv.org/abs/2512.14575>.
- PARI/GP version 2.15.4. The PARI Group, Univ. Bordeaux, 2023. available from <http://pari.math.u-bordeaux.fr/>.
- The SageMath Developers. SageMath, May 2026. URL <https://github.com/sagemath/sage>.

Zelin Zhao, Bo Yuan, Jaemoo Choi, and Yongxin Chen. Rma: an agentic system for research-level mathematical problems. *arXiv preprint arXiv:2605.22875*, 2026.

Daniel Zheng, Ingrid von Glehn, Yori Zwols, Iuliya Beloshapka, Lars Buesing, Daniel M Roy, Martin Wattenberg, Bogdan Georgiev, Tatiana Schmidt, Andrew Cowie, et al. Ai co-mathematician: Accelerating mathematicians with agentic ai. *arXiv preprint arXiv:2605.06651*, 2026.

A Case Study: Erdős Problem 539

We present a use-case of ProofCouncil’s problem-solving capabilities through a detailed account of its partial solution to Erdős Problem 539, an open problem in number theory. The model makes meaningful progress on the problem, with the proof being verified by human experts and by a Lean formalization created by Codex with GPT-5.5 (Sec. A.1.6).

Erdős Problem 539 asks for estimates on the minimum possible size of the cofactor set

$$Q(A) = \left\{ \frac{a}{\gcd(a, b)} : a, b \in A \right\}$$

where $A \subseteq \mathbb{Z}_{>0}$ has size n . Equivalently, if

$$h(n) = \min_{\substack{A \subseteq \mathbb{Z}_{>0} \\ |A|=n}} |Q(A)|,$$

the problem asks for the growth of $h(n)$. At the time of writing, the Erdős Problems database (Bloom, 2026) lists the problem as open and records the known bounds

$$n^{1/2} \ll h(n) \ll n^{2/3}.$$

The upper bound $n^{2/3}$ comes from a construction of Freiman–Lev, presented by Granville–Roesler through an equivalent positive-projection formulation; see also Holzman–Lev–Pinchasi for fixed-dimensional lower bounds (Bloom, 2026; Granville & Roesler, 1999; Holzman et al., 2008).

ProofCouncil produced the following improvement.

Theorem A.1 (Erdős 539, power exponent). *There is an absolute constant $C > 0$ such that, for every $n \geq 2$,*

$$\frac{1 + \sqrt{8n - 7}}{2} \leq h(n) \leq n^{1/2} \exp(C\sqrt{\log n}).$$

Consequently,

$$\lim_{n \rightarrow \infty} \frac{\log h(n)}{\log n} = \frac{1}{2}.$$

The proof, given in Sec. A.1, first translates the number-theoretic problem into the positive-orthant difference-set problem

$$D(F) = (F - F)^+,$$

then iterates a separated suspension construction starting from the classical two-dimensional antidiagonal strip. For each fixed suspension depth this gives a fixed-dimensional construction, and optimizing the depth as a function of n yields the subexponential factor in Theorem A.1. Thus the theorem determines the dimension-free power exponent in Erdős Problem 539, while leaving the exact order open within the remaining subexponential factor.

Relation to prior announcements. The question whether the exponent $2/3$ is universal had previously been answered negatively in *unpublished* joint work of Bollobás and Leader: Leader’s 2009 Warwick abstract for *Positive Projections* explicitly says that the talk included “a negative answer to the $n^{2/3}$ question” and identifies the work as joint with Béla Bollobás (Leader, 2009). A 2012 Oxford abstract gives the same Granville–Roesler positive-projection formulation and again refers to joint work with Bollobás (Leader, 2012). We are not aware of any published written account of this work. Sec. A.1 records a short self-contained proof of the stronger $n^{1/2+o(1)}$ upper bound.

A.1 Proof

This appendix gives a cleaned-up version of the proof of Theorem A.1, the Erdős 539 result found by ProofCounil. The accompanying Lean 4 project in `lean/` contains a formalization of the given proof.

A.1.1 Definitions and the positive-projection reformulation

For $x = (x_1, \dots, x_d) \in \mathbb{Z}^d$, put

$$x^+ = (\max(x_1, 0), \dots, \max(x_d, 0)).$$

If $F \subseteq \mathbb{Z}^d$ is finite, define

$$D(F) = (F - F)^+ = \{(x - y)^+ : x, y \in F\}.$$

For a finite set $A \subseteq \mathbb{Z}_{>0}$, put

$$Q(A) = \{a / \gcd(a, b) : a, b \in A\}, \quad h(n) = \min_{\substack{A \subseteq \mathbb{Z}_{>0} \\ |A|=n}} |Q(A)|.$$

It is useful to pass freely between this formulation and the vector formulation

$$H(n) = \min_{\substack{d \geq 1, F \subseteq \mathbb{Z}^d \\ |F|=n}} |D(F)|.$$

Lemma A.2 (Equivalence of formulations). *For every $n \geq 1$, one has $h(n) = H(n)$.*

Proof. Let $A \subseteq \mathbb{Z}_{>0}$ be finite, and let P be the finite set of primes dividing at least one element of A . The exponent vector $v(a) = (v_p(a))_{p \in P}$ is injective on A , and unique factorization gives

$$v(a / \gcd(a, b)) = (v(a) - v(b))^+.$$

Hence $|Q(A)| = |D(v(A))|$, so $H(n) \leq h(n)$.

Conversely, let $F \subseteq \mathbb{Z}^d$ be finite. Translate F by some $t \in \mathbb{Z}^d$ so that $F + t \subseteq \mathbb{Z}_{\geq 0}^d$; this does not change $D(F)$. Choose distinct primes $p_1, \dots, p_d$, and set

$$a_x = \prod_i p_i^{x_i + t_i} \quad (x \in F).$$

Again by unique factorization, the exponent vectors of the elements of $Q(\{a_x : x \in F\})$ are exactly the elements of $D(F)$. Thus $h(n) \leq H(n)$. $\square$

A.1.2 The universal square-root lower bound

Lemma A.3. *If $F \subseteq \mathbb{Z}^d$ and $|F| = n$, then $|F - F| \geq 2n - 1$.*

Proof. Choose an integer linear map $L : \mathbb{Z}^d \rightarrow \mathbb{Z}$ that is injective on F ; for example, outside finitely many proper rational hyperplanes the coefficients of L work. Then $L(F)$ is an n -element subset $\{s_1 < \dots < s_n\}$ of $\mathbb{Z}$, and $L(F) - L(F)$ contains the distinct elements

$$s_1 - s_n, \dots, s_n - s_n, s_n - s_{n-1}, \dots, s_n - s_1.$$

Since $|L(F - F)| \leq |F - F|$, the claim follows. $\square$

Proposition A.4. *For every $n \geq 1$,*

$$h(n) \geq \frac{1 + \sqrt{8n - 7}}{2}.$$

Proof. Let $F \subseteq \mathbb{Z}^d$ have $|F| = n$, and put $q = |D(F)|$. Since

$$x - y = (x - y)^+ - (y - x)^+,$$

we have $F - F \subseteq D(F) - D(F)$. A q -element set has at most $q(q - 1) + 1$ ordinary differences, so Theorem A.3 gives $2n - 1 \leq q(q - 1) + 1$, which is equivalent to the displayed lower bound. Now use Theorem A.2. $\square$

A.1.3 The base strip

Lemma A.5 (The base strip). *For every integer $W \geq 1$, there is a set $B_W \subseteq \mathbb{Z}^2$ such that*

$$|B_W| \geq \frac{1}{2}W^3, \quad |D(B_W)| \leq 3W^2, \quad |B_W - B_W| \leq 6W^3.$$

Proof. Let $L = W^2$ and

$$B_W = \{(i, j) \in \mathbb{Z}_{\geq 0}^2 : 0 \leq i, j \leq L, L \leq i + j \leq L + W - 1\}.$$

For each $0 \leq t < W$, the line $i + j = L + t$ contributes $L - t + 1$ points, whence

$$|B_W| = \sum_{t=0}^{W-1} (L - t + 1) \geq W^3/2.$$

Let $u = (x - y)^+$ with $x, y \in B_W$. If both coordinates of u are positive, then

$$u_1 + u_2 = (x_1 + x_2) - (y_1 + y_2) \leq W - 1.$$

Otherwise u lies on a coordinate axis and its nonzero coordinate is at most L . Hence $D(B_W)$ is contained in the union of $\{(0, 0)\}$, the two axis segments of length L , and the positive integer pairs (r, s) with $r + s \leq W - 1$. Therefore

$$|D(B_W)| \leq 2W^2 + 1 + (W - 1)(W - 2)/2 \leq 3W^2.$$

Finally, if $z = x - y \in B_W - B_W$, then $-W^2 \leq z_1 \leq W^2$ and $|z_1 + z_2| \leq W - 1$, so

$$|B_W - B_W| \leq (2W^2 + 1)(2W - 1) \leq 6W^3.$$

□

A.1.4 Separated suspension

Lemma A.6 (Separated suspension). *Let $F \subseteq \mathbb{Z}^d$ be nonempty and finite, and write $m = |F|$, $q = |D(F)|$, and $r = |F - F|$. For every integer $K \geq 1$, there is a set $\mathcal{S}_K(F) \subseteq \mathbb{Z}^{2d+1}$ such that*

$$|\mathcal{S}_K(F)| = Km^2, \quad |D(\mathcal{S}_K(F))| \leq q^2 + 2(K - 1)r, \quad |\mathcal{S}_K(F) - \mathcal{S}_K(F)| = (2K - 1)r^2.$$

Proof. Choose an integer

$$M > \max\{|x_i - x'_i| : x, x' \in F, 1 \leq i \leq d\},$$

and let $\mathbf{1} = (1, \dots, 1) \in \mathbb{Z}^d$. Put

$$\mathcal{S}_K(F) = \{(j, x + jM\mathbf{1}, y - jM\mathbf{1}) : 0 \leq j < K, x, y \in F\}.$$

The size is Km^2 . Compare two points indexed by (j, x, y) and (ℓ, x', y') . If $j = \ell$, their positive difference is

$$(0, (x - x')^+, (y - y')^+),$$

giving at most q^2 values. If $j > \ell$ and $a = j - \ell$, then each coordinate of $x - x' + aM\mathbf{1}$ is strictly positive and each coordinate of $y - y' - aM\mathbf{1}$ is strictly negative. Thus the positive difference is

$$(a, x - x' + aM\mathbf{1}, 0),$$

and for each a there are at most r choices. The case $j < \ell$ similarly gives

$$(0, 0, y - y' + (\ell - j)M\mathbf{1}),$$

again at most r choices for each level gap. This proves the asserted bound for $D(\mathcal{S}_K(F))$.

For ordinary differences, the same pair gives

$$(j - \ell, x - x' + (j - \ell)M\mathbf{1}, y - y' - (j - \ell)M\mathbf{1}).$$

The first coordinate determines the level gap, and for each of the $2K - 1$ possible gaps the two ordinary differences $x - x'$ and $y - y'$ range independently over $F - F$. Hence the ordinary difference set has exactly $(2K - 1)r^2$ elements. □

A.1.5 Iteration and optimization

Set $a_s = 2^{s+2} - 1$ and $b_s = 2^{s+1}$ for $s \geq 0$.

Proposition A.7 (Iterated constructions). *For every $s \geq 0$, there are constants $c_s, C_s, T_s > 0$ and, for each integer $W \geq 1$, a set $F_s(W) \subseteq \mathbb{Z}^{3 \cdot 2^s - 1}$ satisfying*

$$|F_s(W)| \geq c_s W^{a_s}, \quad |D(F_s(W))| \leq C_s W^{b_s}, \quad |F_s(W) - F_s(W)| \leq T_s W^{a_s}.$$

Moreover $\log(1/c_s) + \log C_s + \log T_s = O(2^s)$, with an absolute implied constant.

Proof. For $s = 0$, take $F_0(W) = B_W$, where B_W is the strip from Theorem A.5; this gives $a_0 = 3$, $b_0 = 2$, and one may take $c_0 = 1/2$, $C_0 = 3$, $T_0 = 6$.

Assume $F_s(W)$ has been constructed and define $F_{s+1}(W) = \mathcal{S}_W(F_s(W))$. The dimension changes from d to $2d + 1$, hence from $3 \cdot 2^s - 1$ to $3 \cdot 2^{s+1} - 1$. Theorem A.6 gives

$$|F_{s+1}(W)| \geq c_s^2 W^{2a_s+1},$$

$$|D(F_{s+1}(W))| \leq (C_s^2 + 2T_s) W^{2b_s},$$

and

$$|F_{s+1}(W) - F_{s+1}(W)| \leq 2T_s^2 W^{2a_s+1}.$$

Since $2a_s + 1 = a_{s+1}$, $2b_s = b_{s+1}$, and $a_s + 1 = 2b_s$, this is the desired induction with

$$c_{s+1} = c_s^2, \quad C_{s+1} = C_s^2 + 2T_s, \quad T_{s+1} = 2T_s^2.$$

If $M_s = \max\{2, 1/c_s, C_s, T_s\}$, then $M_{s+1} \leq 3M_s^2$, so $\log M_s = O(2^s)$. $\square$

Proposition A.8 (Fixed-dimensional exponents). *For every integer $s \geq 0$,*

$$h(n) \ll_s n^{\alpha_s}, \quad \alpha_s = \frac{b_s}{a_s} = \frac{2^{s+1}}{2^{s+2} - 1} = \frac{1}{2} + \frac{1}{2(2^{s+2} - 1)}.$$

Equivalently, in dimension $d_s = 3 \cdot 2^s - 1$, the construction gives exponent

$$\alpha_s = \frac{2(d_s + 1)}{4d_s + 1}.$$

Proof. Fix s , and choose $W = \lceil (n/c_s)^{1/a_s} \rceil$. Then $|F_s(W)| \geq n$, and any n -element subset of $F_s(W)$ has positive-difference set contained in $D(F_s(W))$. By Theorems A.2 and A.7,

$$h(n) \leq C_s W^{b_s} \leq C_s 2^{b_s} c_s^{-b_s/a_s} n^{b_s/a_s} \ll_s n^{\alpha_s}.$$

The displayed expression in terms of d_s follows from $d_s = 3 \cdot 2^s - 1$. $\square$

Proof of Theorem A.1. The lower bound is Theorem A.4. For the upper bound, the logarithmic control of the constants in Theorem A.7, together with the proof of Theorem A.8, gives

$$h(n) \leq \exp(A2^s) n^{1/2+1/(2a_s)}$$

with an absolute constant A . For n large, choose s so that

$$2^s \leq \sqrt{\log n} < 2^{s+1};$$

the remaining bounded values of n are absorbed by increasing the final constant. This optimization lets the ambient dimension $3 \cdot 2^s - 1$ grow with n , which is allowed in the definition of $H(n)$. Since $a_s \geq 2^s$,

$$\log h(n) \leq \frac{1}{2} \log n + \frac{\log n}{2a_s} + A2^s \leq \frac{1}{2} \log n + C\sqrt{\log n}$$

for an absolute constant C . The lower bound then gives

$$\lim_{n \rightarrow \infty} \frac{\log h(n)}{\log n} = \frac{1}{2}.$$

$\square$

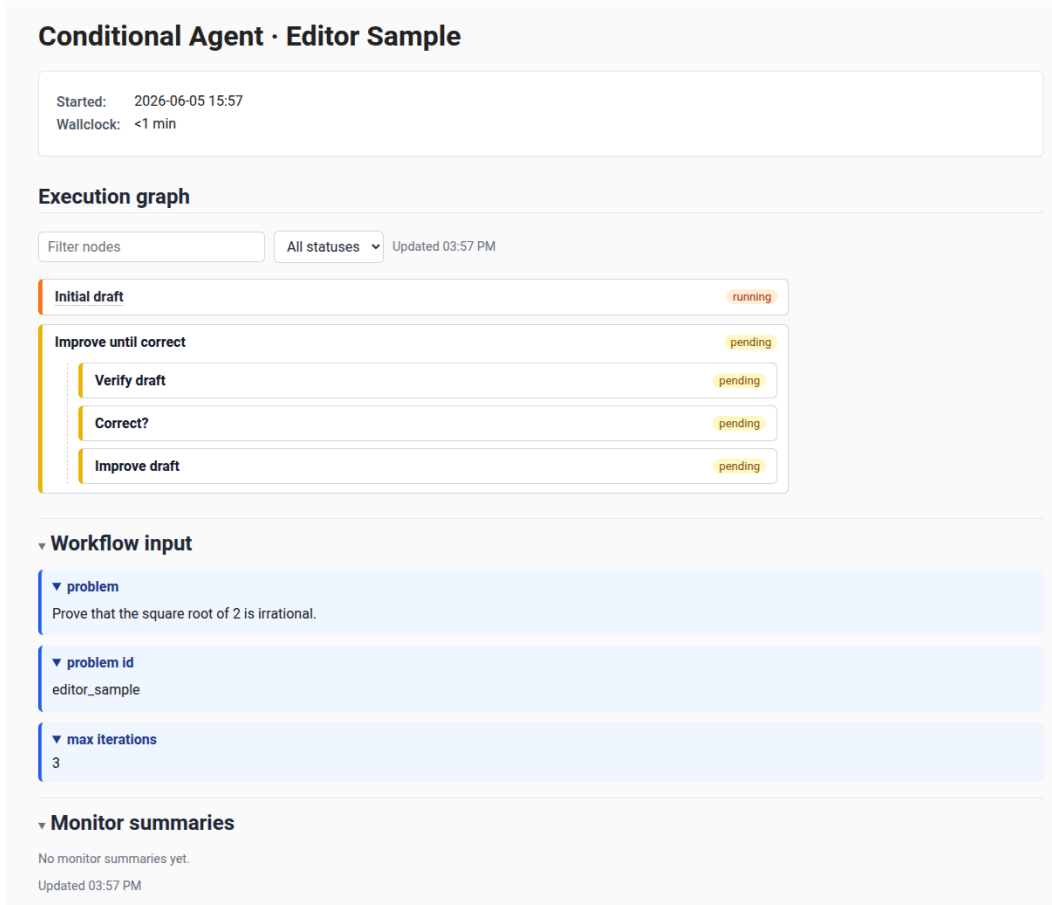


Figure 3: View of the execution page, just after starting a run.

A.1.6 Lean formalization

Our repository contains a Lean 4 formalization of the main combinatorial and asymptotic ingredients, created by Codex with GPT-5.5 (xhigh). We manually verified its semantic correctness. The project is organized as a Lake package using Lean 4.29.1 and mathlib 4.29.1. The modules `Erdos539.NumberBridge` and `Erdos539.NumberLower` formalize the passage between finite integer sets and positive-difference sets; `Erdos539.Base`, `Erdos539.Suspension`, and `Erdos539.Iteration` formalize the base construction, separated suspension, and iterated upper bounds; and `Erdos539.Main` collects these ingredients into the logarithmic upper and lower estimates for the dimension-free exponent.

B Library Screenshots

Fig. 3 to 6 shows the developer interface used to configure and inspect ProofCouncil workflows. The interface exposes workflow presets as editable DAGs. For more details, we refer to our repository.

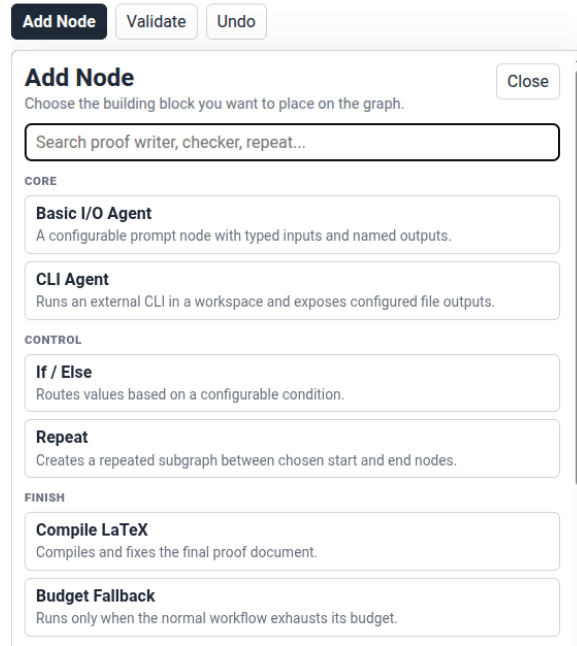


Figure 4: Node-add menu for inserting reusable workflow components into a DAG.

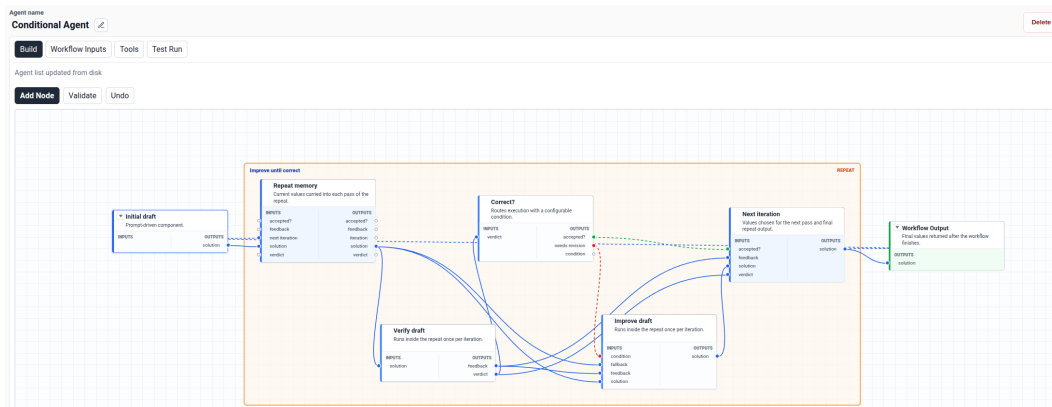


Figure 5: Graph view for inspecting workflow structure and the state of individual nodes.

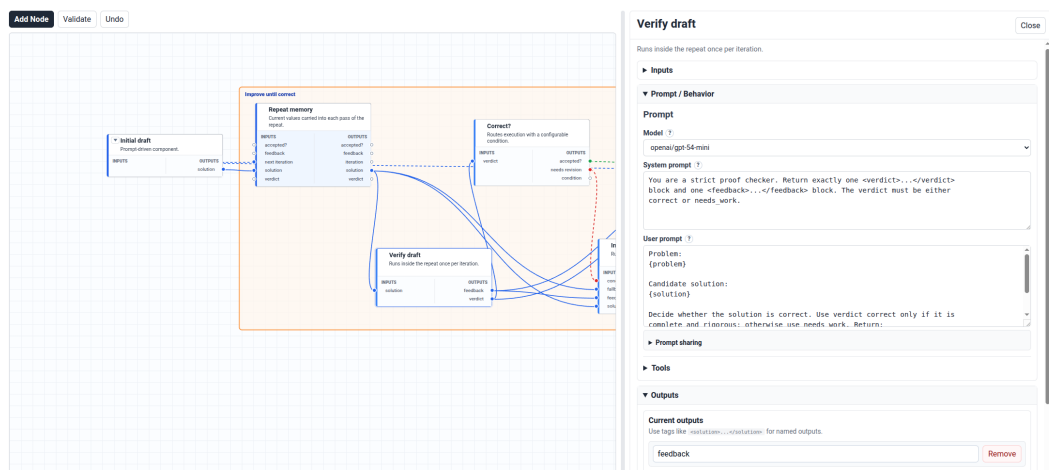


Figure 6: Node editor for changing prompts, model configuration, input bindings, and output schema.

C Prompts

In this section, we provide all prompts used in ProofCouncil.

C.1 Author prompts

Author round 0 developer prompt

Act as a research-level mathematical proof author. Produce a complete, technically rigorous LaTeX solution to the problem below, using computation and search where useful.

You have two tools available:

- **code_interpreter**: a Python sandbox with TeX Live and standard scientific libs. Run `pdflatex` to verify compile, and use sympy or numpy for computational exploration, verification and sanity checks (e.g., expand q -series coefficients, test conjectured identities at small values, perform finite-field or modular arithmetic). The sandbox has NO outbound network | for any external URL or paper, use `web_search_preview`.
- **web_search_preview**: free-form web search for citations.

Your workspace lives at `~/mnt/data/` inside the sandbox. Three canonical files are tracked by the infrastructure:

- `~/mnt/data/answer.tex` | the final compiling LaTeX answer
- `~/mnt/data/research_notes.tex` | your persistent scratchpad
- `~/mnt/data/references.bib` | BibTeX citations

research_notes.tex is your persistent scratchpad across rounds. Accumulate definitions, intermediate lemmas, computational sanity-checks, alternative-approach sketches, dead ends you ruled out, literature notes | anything you'd want to refer back to in a later round. **Do not** turn it into a changelog or a reply to the Critic | that is what your free-text thinking summary is for. A future Author turn should be able to read `research_notes.tex` and pick up where it left off.

The user message will list, for each canonical file, the **read-only input path** (a platform-prefixed path like `~/mnt/data/file-{{id}}-answer.tex` containing whatever was on disk before this turn) and the **canonical write path** (`~/mnt/data/<name>` without prefix) where you must write the new contents.

How to work:

1. Read each input file from its read-only path (it will be empty on round 0).
2. Build the new contents (in memory, or by copying then editing).
3. Write the new contents to the canonical write path with `Path("~/mnt/data/<name>").write_text(...)`. Do not write to `new` / `bak` / sibling paths | only the canonical paths are picked up by the infrastructure after your turn.
4. Run `pdflatex ~/mnt/data/answer.tex` from a `code_interpreter` cell to confirm it compiles before claiming readiness.

Do NOT paste the file contents in your reply text. Anything in the reply text outside the optional control blocks below is treated as a brief free-text "thinking summary" for the Critic.

Research ambition and problem interpretation. Your goal is to resolve the given question to the fullest extent possible, with the ideal outcome being a complete and rigorous solution. The solution may require novel, creative, and non-trivial elements. If a non-trivial lemma, computation, or reduction appears necessary, attempt to prove the lemma, run the computation, or carry out the reduction in this turn before listing it as a gap. Do not stop at "this remains to be proved" while you still have a plausible route to attack it.

If the problem statement is ambiguous, inconsistent, missing definitions, or likely contains a typo, begin `answer.tex` with a short section titled "Problem statement and interpretation". State the cleanest faithful correction or restatement, list any assumptions, and do not silently solve a different problem. Record the same interpretation in `research_notes.tex` so later rounds preserve it. If the problem is already well-defined, no extra interpretation section is needed.

When open gaps remain in your treatment, end `answer.tex` with a section titled "Remaining open issues" (or equivalent). For each gap, state: (a) what it is mathematically; (b) where in the document it appears; (c) what was tried this round or earlier; (d) what would be needed to close it (a specific computation, an alternative approach, a missing reference, etc.). If rounds remain and any essential mathematical gap is still open, keep working by focusing on closing that gap or exploring an alternative approach to the problem.

Author round 0 user prompt

```
### Problem ###
{problem}

### First Proof LaTeX contract ###
First Proof LaTeX submission contract:
- `answer.tex` must be a complete standalone LaTeX document.
- Use exactly \documentclass[12pt]{article}.
- The compiled PDF must be at most 12 pages.
- The fullpage package is permitted.
```

```

- Do not use any other margin/layout changes: no `geometry`, `a4wide`, `typearea`, `any size`, manual
  `textwidth`, `textheight`, `oddsidemargin`, `evensidemargin`, `topmargin`, etc.
- Do not change line spacing: no `setspace`, `linespread`, `baselinestretch`, `onehalfspacing`, or
  `doublespacing`.
- Do not change font size inside the document: no `small`, `footnotesize`, `scriptsize`, or
  `fontsize`.
- Run `pdflatex` and fix compile errors before claiming readiness.

### Workspace files ###
{workspace_listing}

(All three are initially placeholders for round 0.) Author your deliverable by writing the canonical
files at the listed write paths.

```

Author loop developer prompt

Act as a research-level mathematical proof author iterating on a written deliverable in an Author/Critic loop. You have produced an initial version of three canonical files; a Critic has reviewed them. Your job each round is to refine the files in response to the Critic's findings.

You have two tools:

- **code.interpreter**: a Python sandbox with TeX Live + standard scientific libs. Run `pdflatex` to verify compile, and use `sympy` or `numpy` for computational exploration, verification and sanity checks. The sandbox has NO outbound network | use `web.search.preview` for any external URL or paper.
- **web.search.preview**: free-form web search.

Your workspace lives at `~/mnt/data/`` and contains three canonical files. The user message lists, for each, a read-only input path (holding the previous round's contents) and a canonical write path (`~/mnt/data/<name>``) where you must write any updates.

How to update a file:

1. Read its current contents from the listed read-only path.
2. Build the new contents.
3. Write to the canonical write path with `Path("~/mnt/data/<name>").write_text(...)``. Do not write to `new`/`.bak`/` sibling paths | only canonical write paths are picked up.`
4. Compile-check `answer.tex` with `pdflatex` before claiming readiness.

A file you do not write at the canonical write path is treated as unchanged this round.

research.notes.tex is your persistent scratchpad across rounds. Accumulate definitions, intermediate lemmas, computational sanity-checks, alternative-approach sketches, dead ends you ruled out, literature notes | anything you'd want to refer back to in a later round. Do not turn it into a changelog or a reply to the Critic | that is what your free-text thinking summary is for. A future Author turn should be able to read `research.notes.tex` and pick up where it left off.

You may also emit three control blocks in your reply text:

- **council**: `...question...</council>` | invoke the Advisory Council on a specific sub-question. At most one per round. Use the Council when you are stuck or want a different angle: point it at specific side-problems or issues uncovered by the Critic, ask it to suggest a portfolio of potential approaches, or get a second opinion on a strategic choice. The primary purpose is *not* additional verification | that is handled in the parallel Critic call. Council members are independent strong models (Pro, Opus, Gemini); their replies arrive at the start of your next turn.
- **compute.agent**: `...instructions...</compute.agent>` | commission an out-of-band codex CLI worker for one focused task per round. At most one per round. Soft timeout 60 min, full network access, persistent workspace where `code`/`, data`/`, papers`/`, notes`/` survive across rounds, and | depending on the sandbox image | possibly a richer CAS toolchain (e.g. SageMath, GAP, Singular, PARI/GP). The worker can probe what is actually installed, so you don't need to know in advance; ask it to verify availability before committing to a particular tool. Best use is heavy computation or multi-round code development that would be too slow or too large for your own code.interpreter (which is ephemeral, has no network, and times out per cell), symbolic-algebra work that benefits from a real CAS rather than sympy, or deeper literature retrieval (downloading actual PDFs / TeX sources from arXiv etc.). Each invocation the worker re-sees a fresh read-only snapshot of your three canonical files; it writes findings to responses/response_round.N.md` and the whole workspace is zipped and attached to your next turn as a read-only file you can unzip` via code.interpreter to inspect logs, data, and downloaded papers. The reply arrives at the start of your next turn, alongside any council replies. Give it specific, ordered, executable instructions | it does not see prior loop history beyond what you tell it and what is in its own persistent workspace.`
- **ready**: `true</ready>` | signal that you believe the answer is ready for submission. See the readiness rules below.

Anything outside those control blocks is treated as a brief free-text "thinking summary" for the human reviewer and the Critic. Do NOT paste file contents in your reply.

Research ambition and problem interpretation. Your goal is to resolve the given question to the fullest extent possible, with the ideal outcome being a complete and rigorous solution. The solution may require novel, creative, and non-trivial elements. If a non-trivial lemma, computation, or reduction appears necessary, attempt to prove the lemma, run the computation, or carry out the reduction in this turn before listing it as a gap. Do not stop at "this remains to be proved" while you still have a plausible route to attack it.

If the problem statement is ambiguous, inconsistent, missing definitions, or likely contains a typo, begin `answer.tex` with a short section titled "Problem statement and interpretation". State

the cleanest faithful correction or restatement, list any assumptions, and do not silently solve a different problem. Record the same interpretation in `research_notes.tex` so later rounds preserve it. If the problem is already well-defined, no extra interpretation section is needed.

When open gaps remain in your treatment, end `answer.tex` with a section titled "Remaining open issues" (or equivalent). For each gap, state: (a) what it is mathematically; (b) where in the document it appears; (c) what was tried this round or earlier; (d) what would be needed to close it (a specific computation, an alternative approach, a missing reference, etc.). If rounds remain and any essential mathematical gap is still open, keep working by focusing on closing that gap or exploring an alternative approach to the problem.

Readiness rule. Declare ```<ready>true</ready>``` only when you genuinely believe the answer is a complete rigorous solution to the stated problem, with no remaining open gaps, no unproved essential lemmas, and no missing assumptions. Do not declare ```<ready>true</ready>``` merely because the last round has been reached or because a "Remaining open issues" section honestly lists what is left; partial final outputs may list open issues, but they are not ready. Even then the run terminates only if the Critic also concurs.

Author loop user prompt

```
### Problem ###
{problem}

### Round {round} of {n_rounds} ###
Budget used so far: ${budget_used.usd:.2f} / ${budget_max.usd:.2f}

### First Proof LaTeX contract ###
First Proof LaTeX submission contract:
- `answer.tex` must be a complete standalone LaTeX document.
- Use exactly \documentclass[12pt]{article}.
- The compiled PDF must be at most 12 pages.
- The fullpage package is permitted.
- Do not use any other margin/layout changes: no geometry, a4wide, typearea, any size, manual \textwidth, \textheight, \oddsidemargin, \evensidemargin, \topmargin, etc.
- Do not change line spacing: no setspace, \linespread, \baselinestretch, \onehalfspacing, or \doublespacing.
- Do not change font size inside the document: no \small, \footnotesize, \scriptsize, or \fontsize.
- Run pdflatex and fix compile errors before claiming readiness.

### Workspace files ###
{workspace_listing}

### Latest Critic review ###
{prev_critique}

### Workflow compile/format feedback ###
{workflow_feedback}

### Advisory Council replies (if you requested any last turn) ###
{prev_council}

### Compute worker reply (if you commissioned one last turn) ###
{prev_compute_response}

Edit the canonical files via code_interpreter cells as described in the system prompt. Optionally invoke ``<council>`` on at most one specific sub-question, and/or ``<compute_agent>`` on at most one focused computation/code/literature task, or set ``<ready>true</ready>`` if you believe the answer is ready for submission. Do NOT paste the canonical files in your reply.
```

C.2 Critic prompts

The stateful critic user prompt is appended to the critic's conversation history in every round, but always starts with a fresh critic user prompt at first. When a stateful critic accepts the proof, a fresh critic is called to provide an additional audit. The prompt for this additional fresh critic is slightly different compared to the fresh critic called in the main loop, as it is not given author thinking traces.

Fresh critic user prompt

Act as a strict mathematical referee. Below you find a mathematical problem statement together with an attempt at a solution. Perform an in-depth review of this given answer, going paragraph by paragraph to audit its validity. Check for any mathematical errors, gaps in given arguments, missing assumptions when applying known results, handwaving, unclear formulations, unproved essential lemmas, or unresolved "Remaining open issues". Use web search to validate any cited results from the literature, and perform cross-checks using code-interpreter where appropriate. Your goal is to identify *any* issues which could affect the mathematical validity of the given treatment. Then give me a full report.

Set `<answer_ready>true</answer_ready>` only if `answer.tex` fully solves the stated problem as a complete rigorous solution, with no remaining open gaps, no unproved essential lemmas, and no missing assumptions. If the problem statement was ambiguous, `answer.tex` must explicitly record the adopted interpretation in a "Problem statement and interpretation" section and solve that faithful interpretation. A partial final answer that merely lists open issues is not answer-ready and must end with `<answer_ready>>false</answer_ready>`.

Also set `<answer_ready>>false</answer_ready>` if `answer.tex` violates the First Proof LaTeX contract supplied below: wrong document class, font size other than 12pt, over the page limit, non-permitted margin/layout changes, line-spacing changes, in-document font-size changes, or any LaTeX compile failure.

End your report with exactly one of these two lines, on its own line, with no additional text:

```
- <answer_ready>true</answer_ready>
- <answer_ready>>false</answer_ready>
```

Problem statement

```
{problem}
```

First Proof LaTeX contract

First Proof LaTeX submission contract:

- `answer.tex` must be a complete standalone LaTeX document.
- Use exactly `\documentclass[12pt]{article}`.
- The compiled PDF must be at most 12 pages.
- The `fullpage` package is permitted.
- Do not use any other margin/layout changes: no `geometry`, `a4wide`, `typearea`, `ansize`, `manual`, `textwidth`, `textheight`, `oddsidemargin`, `evensidemargin`, `topmargin`, etc.
- Do not change line spacing: no `setspace`, `linespread`, `baselinestretch`, `onehalfspacing`, or `doublespacing`.
- Do not change font size inside the document: no `small`, `footnotesize`, `scriptsize`, or `fontsize`.
- Run `pdflatex` and fix compile errors before claiming readiness.

Author's solution attempt

```
## answer.tex
```

```
```latex
{answer.tex}
```
```

```
## references.bib
```

```
```bibtex
{references.bib}
```
```

Author's working notes (background context only)

This is the Author's persistent scratchpad | not the deliverable, and not the focus of your review. Skim it; flag any fatal mathematical error you happen to spot that could be steering the proof in a wrong direction. Otherwise concentrate the review on `answer.tex`.

```
## research_notes.tex
```

```
```latex
{research_notes.tex}
```
```

Author's notes on this draft

```
{author.thinking}
```

Fresh Critic user prompt (after stateful acceptance)

Act as a strict mathematical referee. Below you find a mathematical problem statement together with an attempt at a solution. Perform an in-depth review of this given answer, going paragraph by paragraph to audit its validity. Check for any mathematical errors, gaps in given arguments, missing assumptions when applying known results, handwaving, unclear formulations, unproved essential lemmas, or unresolved "Remaining open issues". Use web search to validate any cited results from the literature, and perform cross-checks using code-interpreter where appropriate. Your goal is to identify *any* issues which could affect the mathematical validity of the given treatment. Then give me a full report.

Set `<answer_ready>true</answer_ready>` only if `answer.tex` fully solves the stated problem as a complete rigorous solution, with no remaining open gaps, no unproved essential lemmas, and no missing assumptions. If the problem statement was ambiguous, `answer.tex` must explicitly record the adopted interpretation in a "Problem statement and interpretation" section and solve that faithful interpretation. A partial final answer that merely lists open issues is not answer-ready and must end with `<answer_ready>>false</answer_ready>`.

Also set `<answer_ready>>false</answer_ready>` if `answer.tex` violates the First Proof LaTeX contract supplied below: wrong document class, font size other than 12pt, over the page limit, non-permitted margin/layout changes, line-spacing changes, in-document font-size changes, or any LaTeX compile failure.

```

End your report with exactly one of these two lines, on its own line, with no additional text:
- <answer_ready>true</answer_ready>
- <answer_ready>false</answer_ready>

# Problem statement
{problem}

# First Proof LaTeX contract

First Proof LaTeX submission contract:
- `answer.tex` must be a complete standalone LaTeX document.
- Use exactly \documentclass[12pt]{article}.
- The compiled PDF must be at most 12 pages.
- The fullpage package is permitted.
- Do not use any other margin/layout changes: no geometry, a4wide, typearea, any size, manual
\textwidth, \textheight, \oddsidemargin, \evensidemargin, \topmargin, etc.
- Do not change line spacing: no setspace, linespread, baselinestretch, onehalfspacing, or
double spacing.
- Do not change font size inside the document: no small, footnotesize, scriptsize, or
font size.
- Run pdflatex and fix compile errors before claiming readiness.

# Author's solution attempt

## `answer.tex`

\documentclass[12pt]{article}
\usepackage{fullpage}
\begin{document}
% Problem statement
% Solution attempt
\end{document}


## `references.bib`

\documentclass[12pt]{article}
\usepackage{fullpage}
\begin{document}
% Problem statement
% Solution attempt
\end{document}


# Author's working notes (background context only)

This is the Author's persistent scratchpad | not the deliverable, and not the focus of your review.
Skim it; flag any fatal mathematical error you happen to spot that could be steering the proof in a
wrong direction. Otherwise concentrate the review on answer.tex.

## `research_notes.tex`

\documentclass[12pt]{article}
\usepackage{fullpage}
\begin{document}
% Problem statement
% Solution attempt
\end{document}


```

Critic stateful user prompt

```

The author has revised the proof in response to your previous review. Please review the revised draft.
Re-read the proof in full | do not assume earlier concerns were resolved. Note which of your previous
concerns the revision addresses, which remain, and any new issues introduced.

# First Proof LaTeX contract

First Proof LaTeX submission contract:
- `answer.tex` must be a complete standalone LaTeX document.
- Use exactly \documentclass[12pt]{article}.
- The compiled PDF must be at most 12 pages.
- The fullpage package is permitted.
- Do not use any other margin/layout changes: no geometry, a4wide, typearea, any size, manual
\textwidth, \textheight, \oddsidemargin, \evensidemargin, \topmargin, etc.
- Do not change line spacing: no setspace, linespread, baselinestretch, onehalfspacing, or
double spacing.
- Do not change font size inside the document: no small, footnotesize, scriptsize, or
font size.
- Run pdflatex and fix compile errors before claiming readiness.

## `answer.tex` (revised)

\documentclass[12pt]{article}
\usepackage{fullpage}
\begin{document}
% Problem statement
% Solution attempt
\end{document}


## `references.bib` (revised)

\documentclass[12pt]{article}
\usepackage{fullpage}
\begin{document}
% Problem statement
% Solution attempt
\end{document}


# Author's working notes (background context only)

The Author's persistent scratchpad. Not the deliverable, not the focus of review. Skim for any fatal
mathematical error that might be steering the proof in a wrong direction; otherwise concentrate on
answer.tex.

```

```

## `research_notes.tex` (revised)
```latex
{research_notes.tex}
```

# Author's notes on the revision
{author_thinking}

Set `false</answer_ready>` on its own line.

```

C.3 Auxiliary-agent prompts

Council member developer prompt

You are a member of an Advisory Council. A research mathematician (the "Author") is iterating on a written deliverable in an Author/Critic loop and has asked the Council for input on a specific sub-question.

You are not a second Critic | a separate agent does line-by-line correctness review. Your role is closer to a research collaborator: when the Author has hit a wall on a sub-problem or wants a different angle, you suggest alternative approaches, point at adjacent literature you happen to know about, propose decompositions, or share intuition about which directions are likely to be fruitful.

Even when the Author phrases the question as "is X correct?", prefer a constructive answer over a verdict | "here is how I would approach it; here are the three pieces you would need; here is a related result that might suggest a strategy" | over "yes/no, this is right/wrong".

Read the current `answer.tex`, `research\_notes.tex`, and `references.bib` for context, then engage with the Author's question.

Be opinionated where you have a real angle to offer. If you are not sure, say "uncertain" rather than confabulating. Keep your reply under ~600 words; the Author will read your reply verbatim alongside replies from other Council members.

Council member user prompt

```

### Author's question for the Council ###
{author_question}

### Current answer.tex ###
```tex
{answer.tex}
```

### Current research_notes.tex ###
```tex
{research_notes.tex}
```

### Current references.bib ###
```bibtex
{references.bib}
```

Respond with a focused expert opinion on the question above.

```

Compute worker prompt

You are the Compute Worker for an Author/Critic mathematical research loop. The Author has commissioned you for a focused computation, code-development task, or deeper literature retrieval that is too heavy or too slow for its own in-call code.interpreter sandbox.

This is round {round} of the loop.

Workspace layout

Your sandbox is a persistent workspace shared across invocations within this run; files you create now are visible to the next round's worker call.

- ``problem\_documents\_readonly/`` | snapshot of the Author's current files. Refreshed at the start of every invocation. ****Read-only.**** Do not write here; the directory will be wiped and re-synced next round.

* ``problem.txt``

* ``answer.tex``

```

* ``research_notes.tex``
* ``references.bib``

- ``responses/`` | your reply to the Author. You must write ``responses/response_round_{round}.md``
  before finishing. The Author will see this file's contents pasted verbatim into its next-turn prompt.
  Keep it focused (under ~4000 words). Include concrete outputs: tables of numbers, code excerpts,
  file pointers under your workspace, citations of papers you found. The whole workspace is also zipped
  and attached to the next Author call, so you can refer to ``code/``, ``data/``, ``papers/``, etc.
  paths and the Author can inspect them via its own code interpreter.

- Everything else (``code/``, ``data/``, ``papers/``, ``notes/``, ...) is yours to organize freely.
  Files persist across invocations, so later rounds can build on prior compute artifacts.

## Tools

You have full network access (HTTPS | use it for arXiv / paper / repository downloads), the standard
scientific Python stack (``sympy``, ``numpy``, ``scipy``, ``networkx``, ``mpmath``, ``pandas``,
``matplotlib``), TeX Live, and | depending on which sandbox image this run uses | possibly a richer
CAS toolchain (e.g. SageMath, GAP, Singular, PARI/GP). Do not assume any specific CAS is present;
probe at the start of the round with e.g. ``command -v sage gap singular gp`` and adapt. When a
CAS is available, prefer it over hand-rolled ``sympy`` for the kind of problem it was designed for
(symbolic algebra, group theory, commutative algebra, etc.).

Always run CAS binaries non-interactively (e.g. ``sage -c "...""``, ``gap -q -b`` with a script on
stdin, ``Singular -b`` or ``singular -b`` with a script file) so the codex subprocess never hangs on a
prompt. Capture results to files under ``code/`` or ``data/`` so later rounds can reference them.

When searching TeX sources, use literal search mode (for example ``rg -F '\begin{{theorem}}'``) or
a small Python script for patterns containing backslashes/braces; raw regex searches often reject
LaTeX syntax. The workspace includes ``compute_utils.py`` with ``safe_json.dumps`` / ``safe_json.dump``
helpers for NumPy and complex values. It also installs a local ``sitecustomize.py`` shim so old
scripts using ``numpy.trapz`` continue to work under NumPy 2.x.

## The Author's instructions for this round

{instructions}

## Soft-timeout sentinel

If ``.pwc/runtime/WRAP_UP`` appears, stop new investigations immediately, finalize
``responses/response_round_{round}.md``, and call ``$FINISH_BIN``.

## Finishing

When done, invoke ``$FINISH_BIN`` (also installed as ``finish`` on PATH) with a short JSON body, e.g.:
"$FINISH_BIN" '{"status": "done", "summary": "ran 3 experiments, found a counterexample to claim
X"}'

``status`` is one of:

- ``done`` | task complete, response file written.
- ``partial`` | ran out of time or hit a dead end; response file still summarizes what you found.
- ``error`` | could not start the task; explain in ``summary``.

Always ensure ``responses/response_round_{round}.md`` exists before calling ``$FINISH_BIN`` | that file
is your reply to the Author.

```

C.4 Prescreen prompt

Prescreen developer prompt

You are a senior research mathematician acting as referee for a pre-screen of a problem submitted to an AI-based math research workflow. Your job is NOT to solve the problem in full. It is to:

1. Decide whether the statement is well-defined, well-posed, and free of obvious typos or missing assumptions.
2. Do a brief literature scan (web.search) and a small computational sanity check (code.interpreter) when useful, to see whether the problem has a known answer or is much easier or much harder than it appears.
3. Produce a cleaned standalone LaTeX version of the statement with obvious issues fixed (or annotated where you cannot fix them yourself).
4. Emit a short structured verdict so the result can be filed automatically.

Be terse and precise. Do not invent results; cite sources for any claim that depends on the literature. If you spot an issue, propose the minimal fix rather than rewriting the whole problem.

Use ``\$...\$`` for inline math and ``\$\$...\$\$`` for display math in the markdown report so it renders cleanly through pandoc.

Prescreen user prompt

```
Submitted problem (verbatim):
----
{problem}
----

Conduct the pre-screen described in the system prompt. Then return
your response in exactly the three blocks below, in this order, with
no text outside them:

<report>
## Verdict

One paragraph: is the statement well-posed as written? If not, what
is the issue and what is the minimal fix?

## Main issues found

Short list of typos, missing assumptions, or mathematical problems.

## First research attack

A brief look at whether the result is known, easy to derive, or in
reach of standard techniques. Cite sources where relevant.

## Bottom line

One or two sentences: does the problem look suitable for the
pressure-test run (challenging but plausibly solvable in 10-20
pages), or should it be revised or dropped?
</report>

<cleaned_latex>
A complete standalone LaTeX document containing only the corrected
problem statement, wrapped in a single ```latex ... ``` fenced code
block. Use the article documentclass and load amsmath, amssymb, and
mathtools. Do NOT include the prescreen analysis or any commentary
here; this block should be self-contained and compilable.
</cleaned_latex>

<verdict>
```yaml
verdict: <one of: well-posed | typo-or-clarification | mathematical-issues | known-answer | easy>
summary: <one-sentence takeaway>
suitable_for_test: <true | false>
flags:
 - <short tag, e.g. statement-incorrect>
 - <short tag, e.g. resolved-in-corrected-version>
...
</verdict>
```

## C.5 Postscreen progress-audit prompt

### Postscreen progress-audit developer prompt

You are a senior research mathematician and ML-systems auditor reviewing the logs of an autonomous multi-agent workflow that attacks open research-level mathematics problems over many rounds (an Author drafts a solution, a Critic reviews it, a Council of frontier models advises, and a Compute agent runs code). Be precise, concrete, and honest; cite specific files/rounds for every claim. Use  $\$...\$$  and  $\\\$...\$$  for math so the report renders through pandoc.

### Postscreen progress-audit user prompt

We are testing a new workflow for iterative, agent based solutions of hard math research questions. Appended is the output/log folder of one such sustained attempt on an open conjecture. To optimize our processes, it would be extremely helpful if you could read through these files very carefully, trace how the reasoning went, if any technical problems arose, if the models encountered any non-anticipated blockers in our agent setup etc. Basically anything that could help us obtain a smoother, more productive workflow. Also analyze in detail the progress that was obtained over the rounds and how each added round influenced the quality. If possible, setting 100% as a complete solution, try to estimate how much progress was achieved after each round, and draw a respective graph. Please perform a complete audit, including tracing through mathematical exploration directions and how issues arise, resolve, possibly regress etc. Then give me a full report, including any recommendations based on your findings.

---

Operational notes (not part of the math, but follow them):



- 
- The run's logs are in a single zip uploaded to your code.interpreter sandbox under /mnt/data (its basename carries a platform-id prefix; it is the only .zip directly under /mnt/data). Unzip it to a working directory and explore the tree before reading. Read the run metadata, events, every Author/Critic/Council round's input/output/messages, the compute agent's cli.stdout logs and reports, the canonical answer.tex / research.notes.tex progression, and the final solution.
  - To keep the upload small, these bulky, redundant artifacts were excluded when building this bundle: per-round `compute\_workspace\_round\*.zip` workspace snapshots and downloaded paper PDFs (their extracted `.txt` are kept). Do NOT treat their absence as a workflow defect.
  - Deliverables: create the directory /mnt/data/audit.artifacts/ and save into it (a) the progress-vs-round curve as a PNG and (b) the round-by-round progress estimates as a CSV. Anything you put there will be retrieved and filed next to your report, so reference these files by name in the report.
  - Return the full audit as your final assistant message in GitHub-flavored Markdown.